

# Health Data Sources II: Wearable Sensors and Measurements

+ Week 03

Dr. Xuhai “Orson” Xu | Dr. Eugene Wu

**AI @ VP&S Initiative**



VAGELOS COLLEGE OF  
PHYSICIANS AND SURGEONS

COLLABORATORY  
AT COLUMBIA UNIVERSITY



# Today's Outline

- 1. Motion Sensors and Step Counting**
- 2. PPG Sensor, Heart Rate, and HRV**
- 3. Sleep Stages and Metrics**
- 4. Time Processing and Data Quality**
- 5. Wearable Data Access: APIs and OAuth**

# What's Inside a Wearable Device?

## Fitbit Sense, skin-facing side.

- Skin-facing windows carry light to and from tissue
- Motion sensors sit inside the case

How do these signals become steps, pulse, and sleep estimates?



# Inside a Fitbit Sense

- **Lower left:** housing and optical assembly
- **Upper middle:** circuit board
- **Upper left:** battery

Motion sensors and the processor are small chips inside the case. Software turns sensor readings into metrics.



# A Dashboard Number Hides a Pipeline

```
Physical phenomenon -> sensor -> digital samples -> time index  
-> quality checks -> filtering -> window -> features -> model  
-> displayed metric -> API record
```

Every arrow encodes a choice that can change the result.

# Motion Sensors and Step Counting

Measurements of movement and rotation support orientation estimates and step counts.

# Motion Sensors

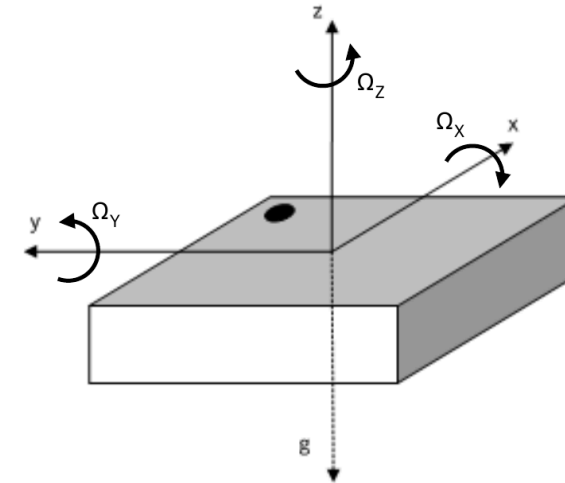
An **IMU (inertial measurement unit)** combines:

- **Accelerometer:** acceleration along x/y/z, including gravity
- **Gyroscope:** angular rate around x/y/z; how fast the device turns ( $^{\circ}/s$ )

**Six-axis = 3 acceleration + 3 gyro channels.**

An optional **magnetometer** measures magnetic field to help estimate heading.

The accelerometer does not directly measure speed or position.



Definition of sensing axes orientation

# Fusion Estimates Gravity and Motion

**Linear acceleration** is a change in speed or direction.

**Orientation** describes tilt and heading.

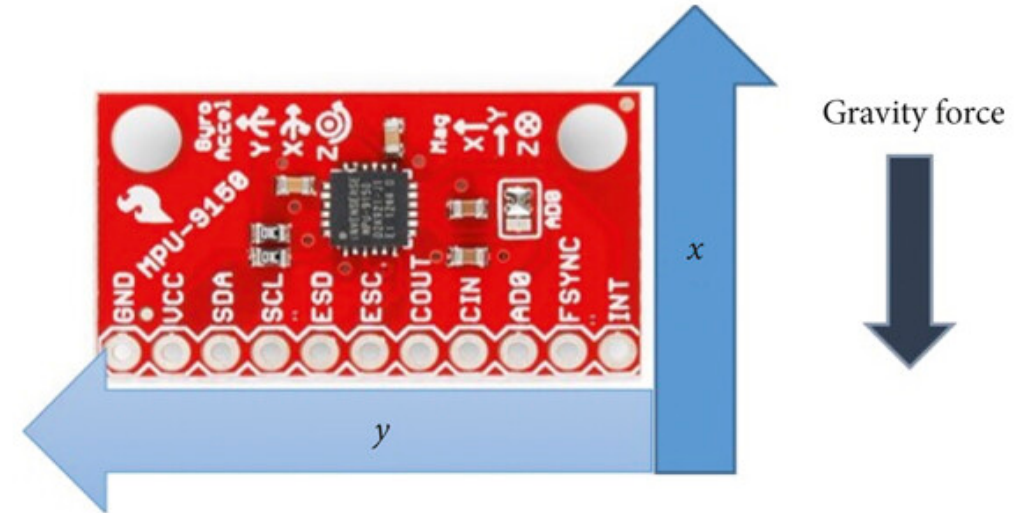
**Sensor fusion** combines sensor measurements:

- Gyroscope predicts orientation changes
- Usable gravity and magnetic references correct them
- Subtract estimated gravity to estimate linear acceleration

**Tilt in place:** gravity shifts across axes.

**Start/stop sliding:** linear acceleration appears.

At rest, gravity's magnitude stays near **1 g**.



# Magnitude Reduces Orientation Dependence

**Magnitude** combines the three acceleration axes:

$$\text{Magnitude} = \sqrt{a_x^2 + a_y^2 + a_z^2}$$

For (0.7, 1.0, 0.6) g:

$$= \sqrt{0.49 + 1.00 + 0.36}$$

$$= \sqrt{1.85} = 1.36 \text{ g}$$

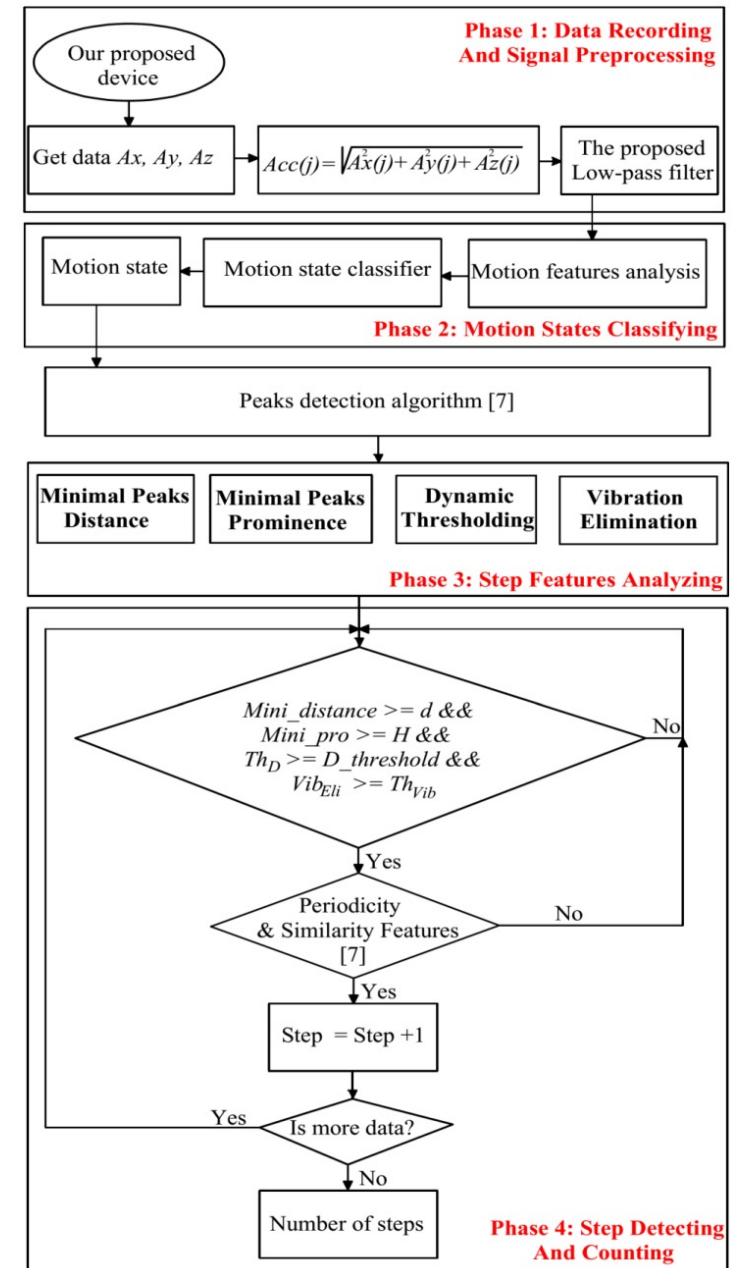
Magnitude retains gravity and discards direction.

# A Rule-Based Step Counter

Before the machine learning era...

A rule-based counter makes explicit choices about which motion patterns count as steps:

1. Combine or orient acceleration axes
2. Reduce gravity and irrelevant frequencies
3. Detect candidate peaks with explicit rules
4. Require plausible cadence or persistence

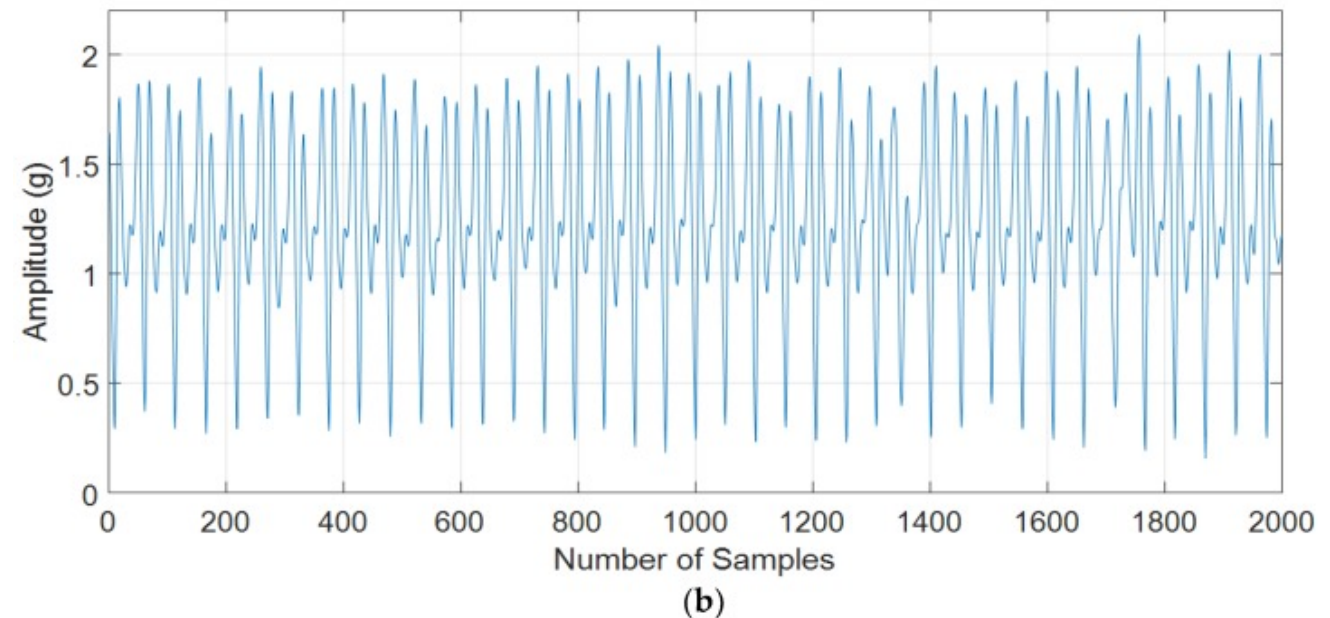
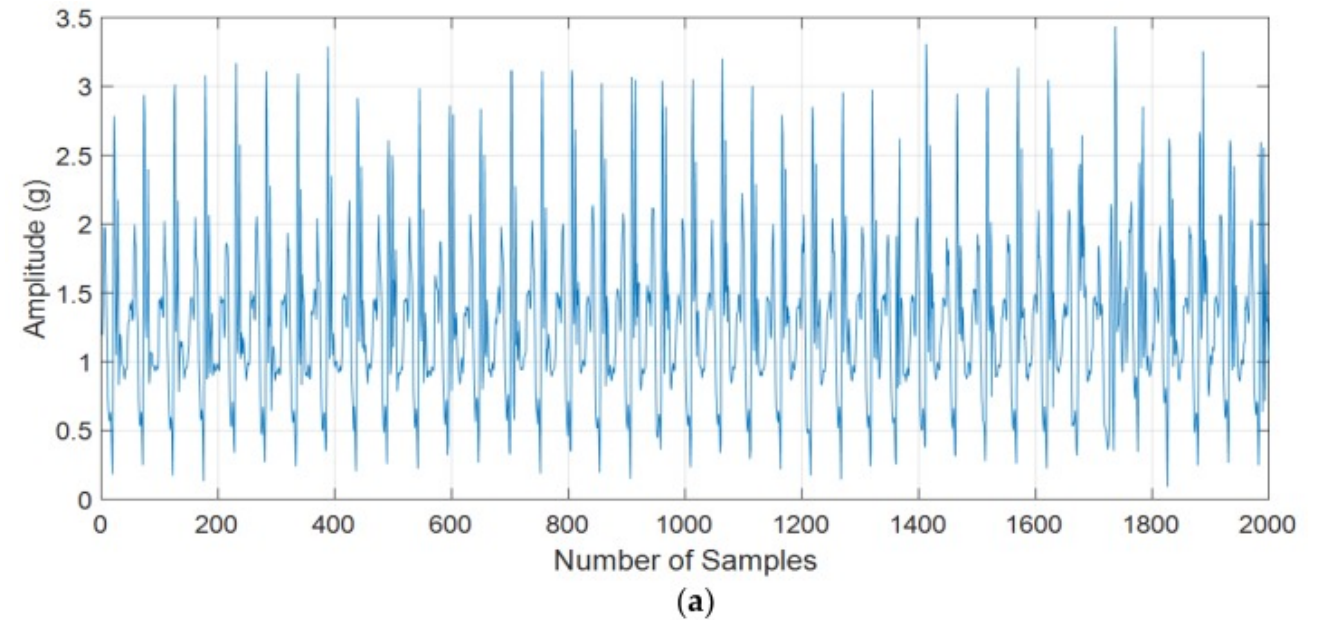


# Filters Decide Which Rhythms Survive

A **filter** reduces selected patterns in a signal.

- High-pass filtering can reduce slow gravity and drift
- Band-pass filtering can retain a chosen cadence range
- Low-pass filtering can suppress selected rapid variation

Inspect the transformed trace before counting peaks.

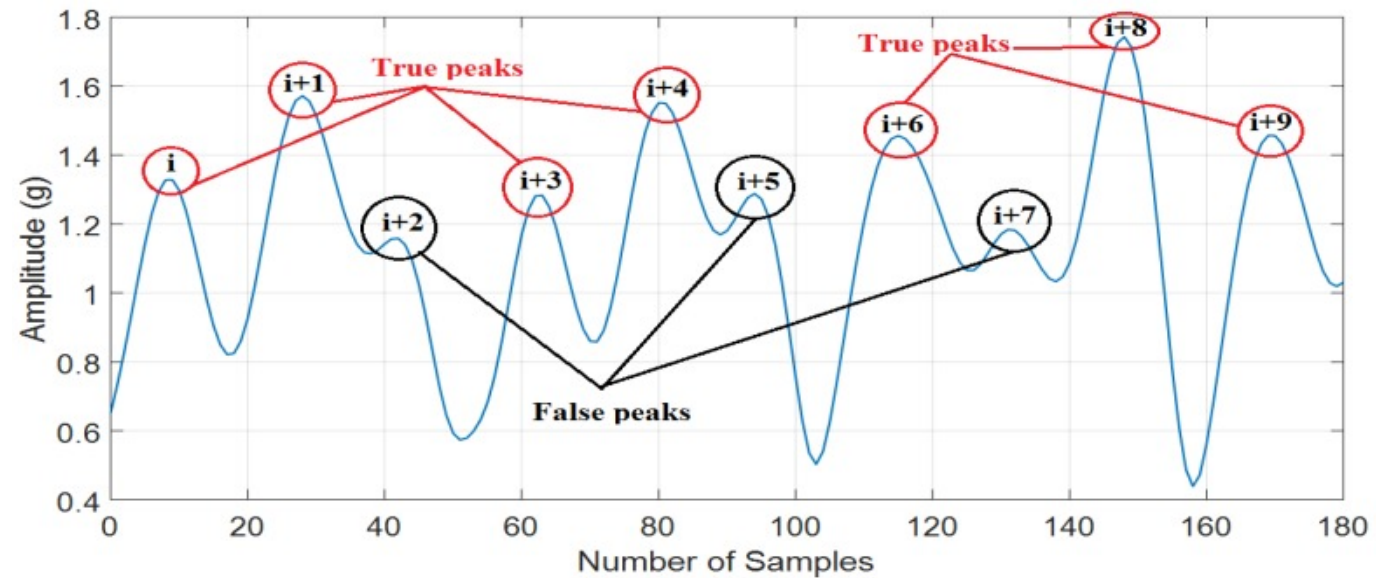


# A Peak Is a Candidate

**Peak detection** finds local high points and selects those meeting a rule.

- **Prominence:** rise above the surrounding baseline
- **Spacing:** minimum separation between retained events
- Stricter rules can reject noise and miss real steps

A retained peak still needs validation as a step.



# Peak Rules Keep Four of Six Candidates

Teaching rule: scan in time order; keep prominence  $\geq 0.20$  g and spacing  $\geq 0.40$  s from the last retained peak.

| Time   | Prominence | Decision          |
|--------|------------|-------------------|
| 0.50 s | 0.31 g     | keep              |
| 1.02 s | 0.28 g     | keep              |
| 1.54 s | 0.12 g     | reject: too small |
| 1.70 s | 0.35 g     | keep              |
| 2.08 s | 0.33 g     | reject: too close |
| 2.60 s | 0.29 g     | keep              |

**Output: 4 candidate steps.**

# Step Counting Errors

| Scenario                     | What happens                                               | Error                                                        |
|------------------------------|------------------------------------------------------------|--------------------------------------------------------------|
| Walking while pushing a cart | Reduced wrist motion can make the counter miss a real step | <b>False negative:</b> a real step is missed                 |
| Sitting on a vibrating bus   | The counter can mistake vibration peaks for steps          | <b>False positive:</b> a step is counted without a real step |

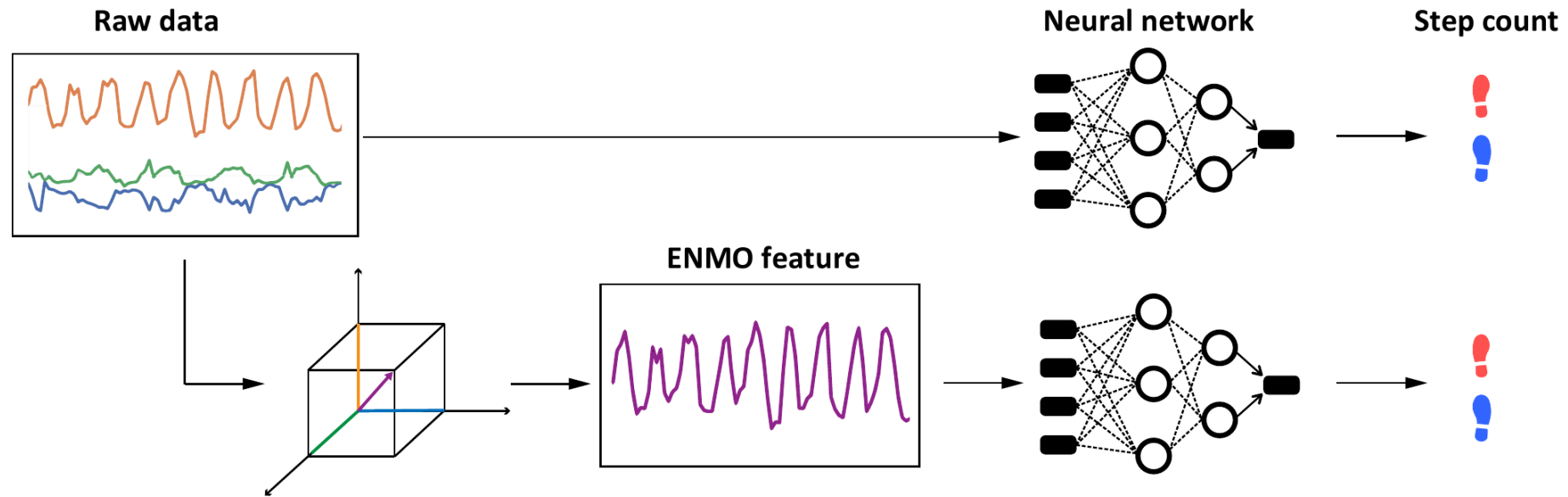
Periodicity alone does not establish walking.

Stricter rules can reject vibration but miss real steps when gait or wrist placement changes.

# Neural Networks Learn Step Patterns

**Deep learning** trains layered **neural networks** to learn patterns across time.

**a**



Input: x/y/z acceleration, or their magnitude; Output: estimated steps.

# Training and Output Are Design Choices

Train on acceleration windows synchronized with **reference labels**: observed steps or walking periods.

| Model output          | Toy output     | Counting step                               |
|-----------------------|----------------|---------------------------------------------|
| Walking / not walking | <b>Walking</b> | A separate counter detects and counts steps |
| Direct count estimate | <b>4 steps</b> | Combine windows without double-counting     |

The open-source OxWearables stepcount tool combines a learned walking detector with peak counting.

State-of-the-art: leveraging IMU foundation models for step counting.

Learning can reduce manual feature design; preprocessing, labels, and counting rules still matter.

# Choosing a Step Counter

| Decision                       | Explicit rules                    | Neural model                            |
|--------------------------------|-----------------------------------|-----------------------------------------|
| Data                           | Examples to tune and test         | Representative labels to train and test |
| Inspection                     | Trace thresholds and filters      | Inspect predictions and failure cases   |
| New people, gait, or placement | Recheck against a reference       | Test transfer; retrain if needed        |
| Device resources               | Measure cost of filters and rules | Measure cost of model inference         |

Learning is useful when representative labeled data are available. Acquiring diverse data remains a cost (getting lower and lower in recent years).

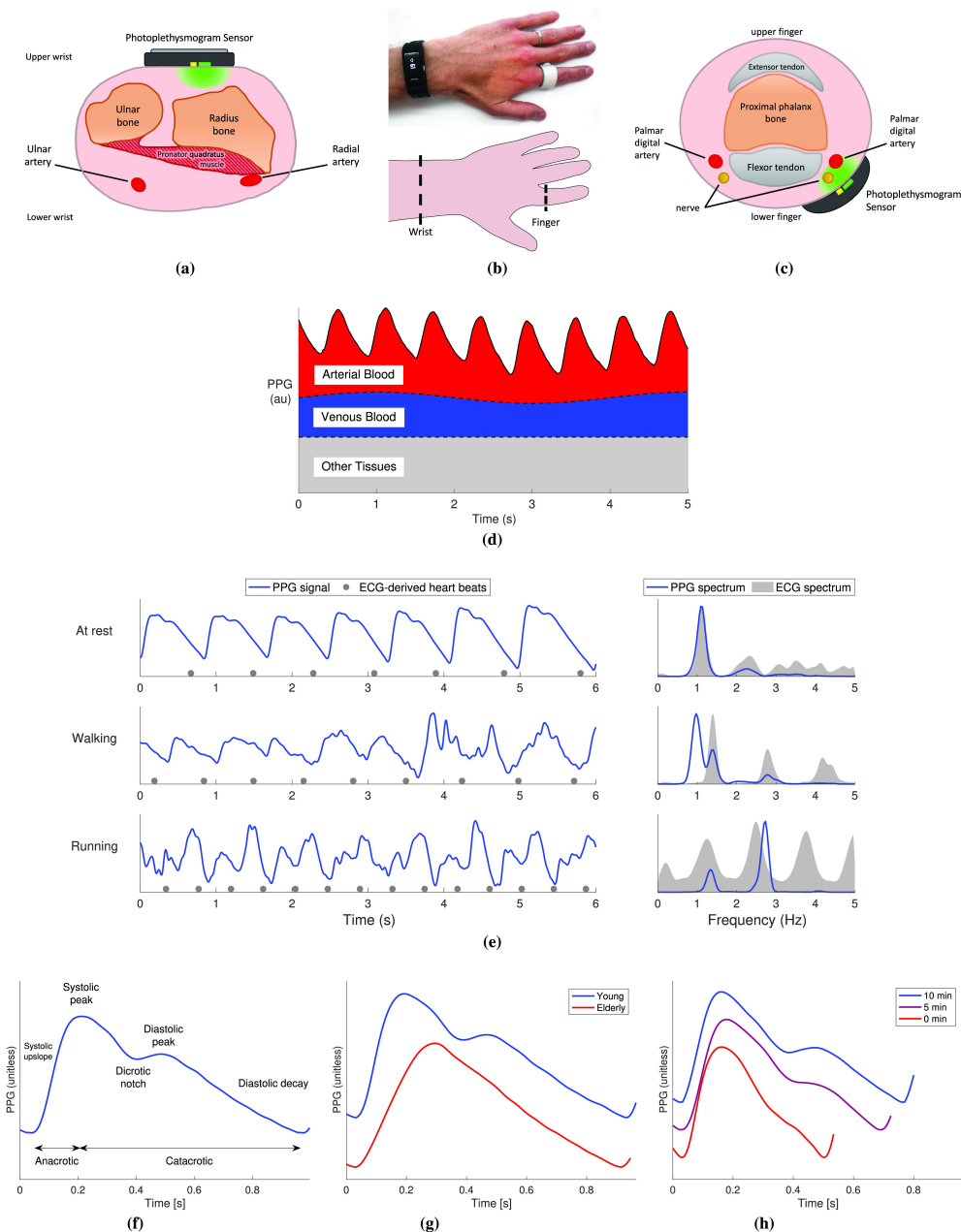
# PPG Sensor, Heart Rate, and HRV

An optical waveform supports pulse timing and interval variability.

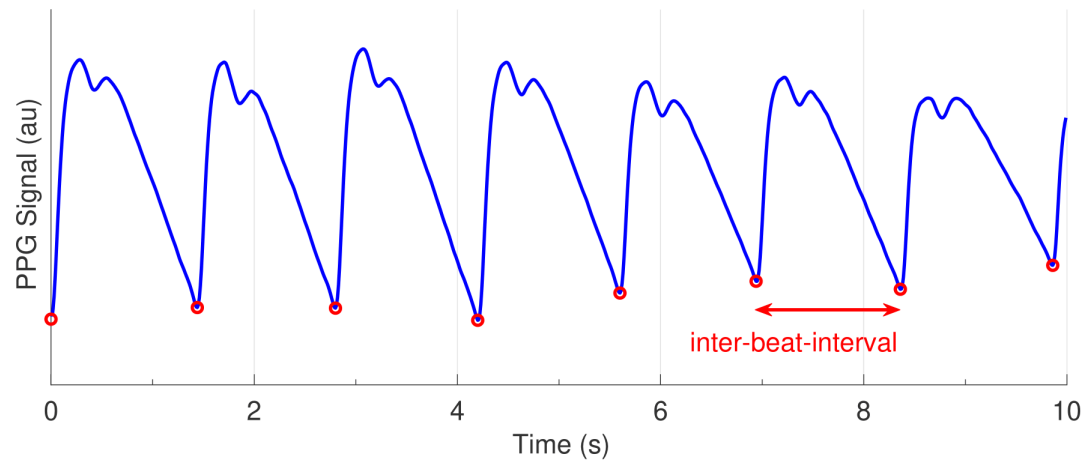
# PPG Reads Peripheral Blood-Volume Change

**Photoplethysmography (PPG)** measures an optical signal affected by peripheral blood-volume change.

- Light-emitting diodes illuminate tissue; a detector measures returned light
- Peaks can support pulse-interval estimates
- An electrocardiogram (**ECG**) measures cardiac electrical activity; PPG does not



# A Waveform Becomes a Pulse Estimate



Intervals: 0.83, 0.80, 0.87 s

Mean = 0.833 s

Pulse rate =  $60 / 0.833$   
= 72.0 bpm

Missed or extra peaks change the intervals and the displayed rate.

PPG provides peripheral pulse timing; ECG events and diagnoses require different evidence.

# Motion Can Mimic the Pulse

During activity, wrist motion changes contact and the optical path.

Target pulse rhythm: 1.5 Hz

Motion rhythm: 1.5 Hz

When the artifact overlaps the target frequency, filtering by frequency alone cannot separate them. Concurrent accelerometry and signal-quality checks can support artifact rejection, but the result remains method-dependent.

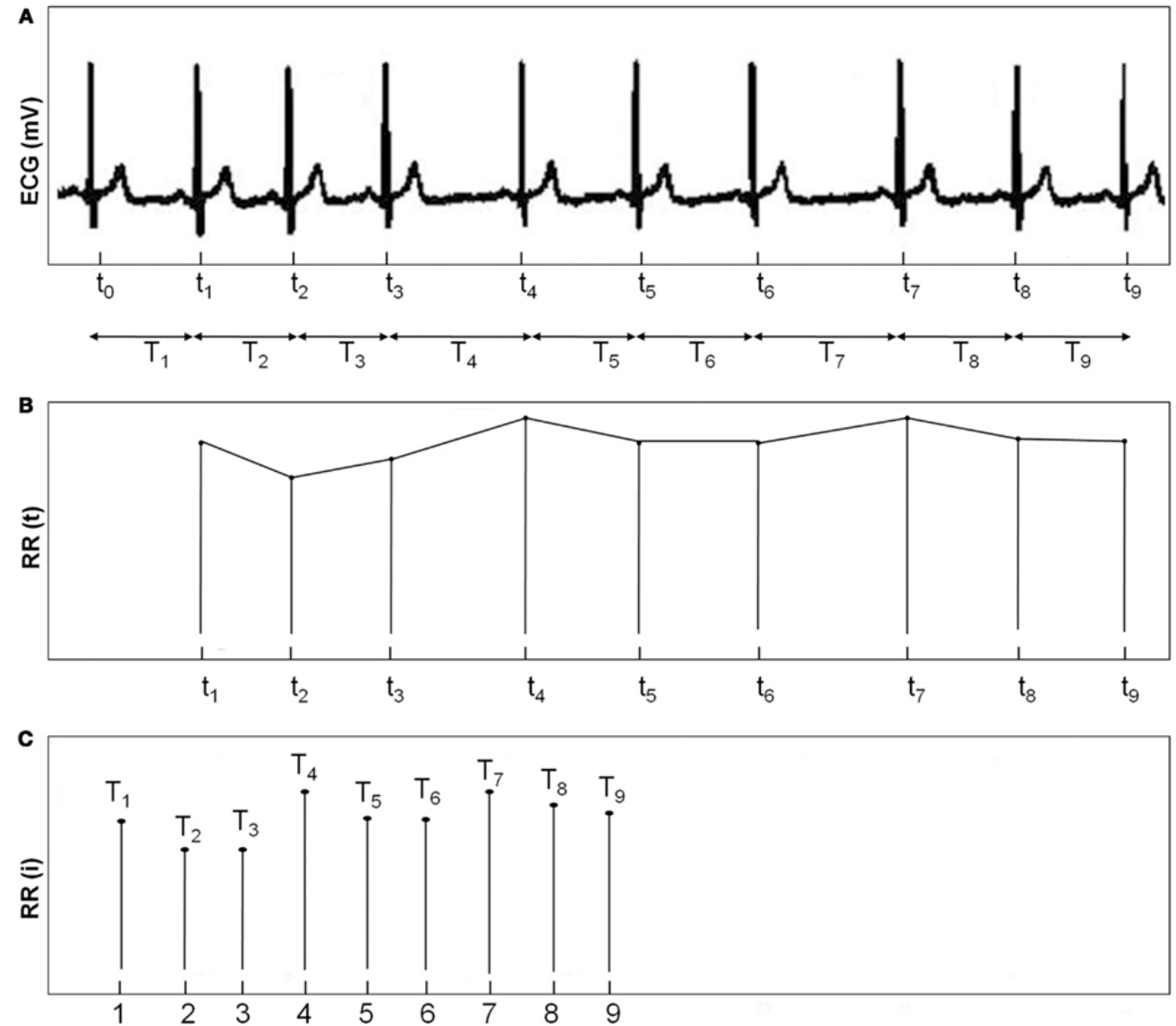
# HRV Begins with Beat Intervals

**Heart-rate variability (HRV)** describes variation in the time between heartbeats.

An **NN (normal-to-normal) interval** spans successive normal beats, measured between ECG R peaks.

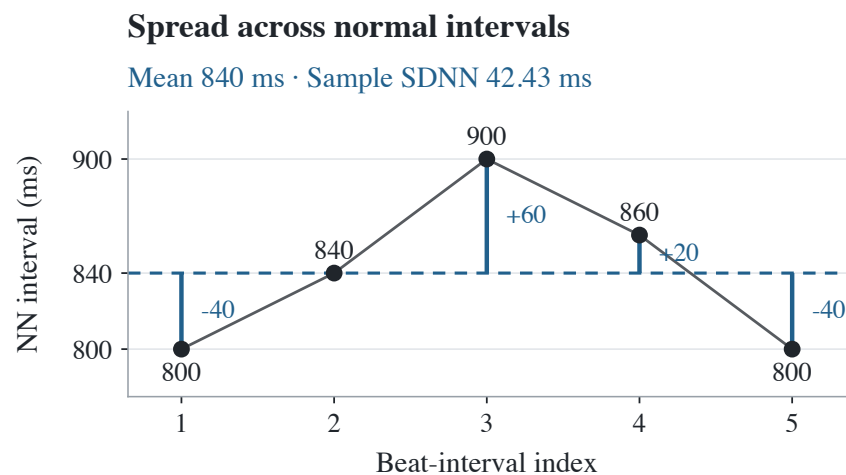
Detect beats, handle artifacts, then compare intervals.

Wrist PPG uses pulse timing; its HRV estimates can differ from ECG.

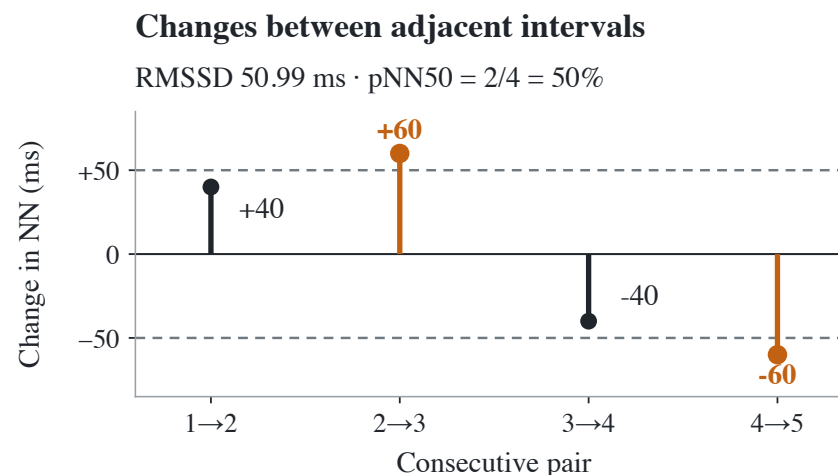


# Common HRV Features Describe Interval Variation

- **SDNN**: sample standard deviation of NN intervals
- **RMSSD**: root mean square of successive NN differences
- **pNN50**: percentage of adjacent NN pairs differing by **more than 50 ms**
- **LF / HF**: power in low- and high-frequency bands, and their ratio



Synthetic teaching example · Orange:  $|\text{change}| > 50$  ms, for 2 of the 4 consecutive pairs



Established Python packages compute these features from a list of IBIs: **Neurokit2** and **HeartPy**

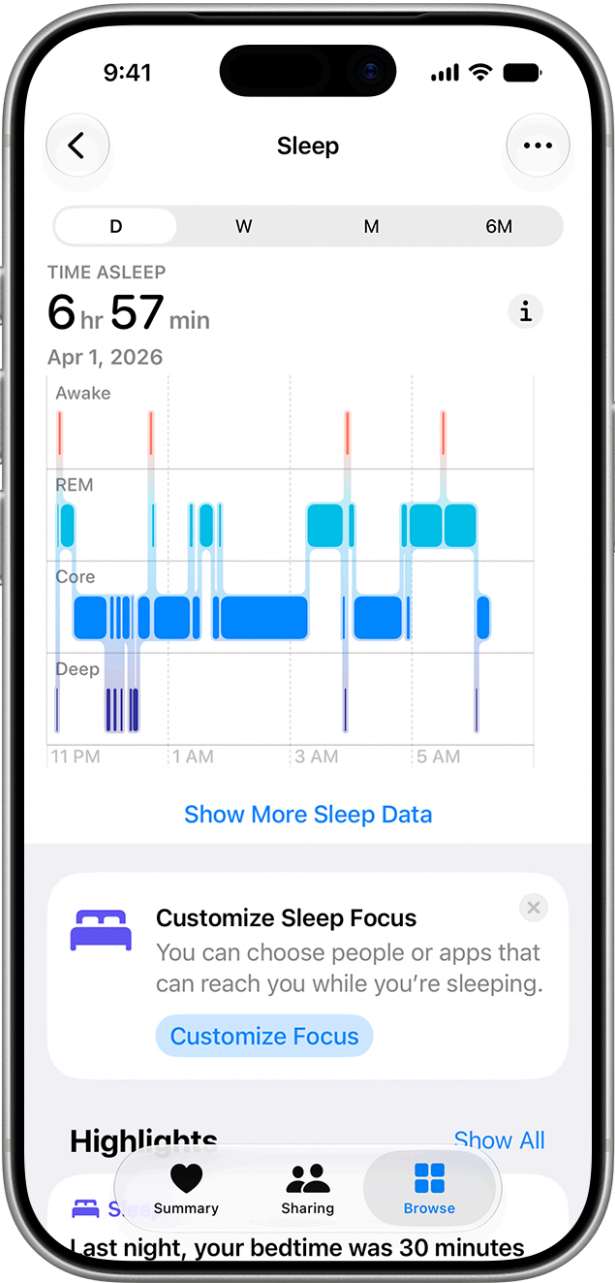
# Sleep Stages and Metrics

Motion and pulse patterns support estimates of sleep stages and duration.

# Sleep Stages in a Wearable App

| Clinical stage | Meaning               | label                                 |
|----------------|-----------------------|---------------------------------------|
| N1             | Sleep onset           | Core (Apple) /<br>Light (Fitbit/Oura) |
| N2             | Established sleep     | Core (Apple) /<br>Light (Fitbit/Oura) |
| N3             | Slow-wave sleep       | Deep                                  |
| REM            | Rapid eye<br>movement | REM                                   |

The screen shows estimated stages and awake intervals.



# Sleep Duration and Efficiency

Teaching night: in bed and trying to sleep at **23:00**; final awakening and getting up at **07:00**.

- **Sleep latency:** 20 min from trying to sleep to sleep onset
- **Wake after sleep onset (WASO):** 40 min awake between sleep onset and final awakening

Time in bed = 480 min

Total sleep time = 480 - 20 - 40 = 420 min (7 h)

Sleep efficiency = total sleep time / time in bed × 100  
= 420 / 480 × 100 = 87.5%

# Polysomnography as Ground Truth

**Polysomnography (PSG)** records several body signals during sleep. Much richer than PPG. For example:

- **EEG:** brain electrical activity
- **EOG:** eye movements
- **Chin EMG:** muscle electrical activity

These signals support sleep-stage scoring. PSG also monitors breathing and oxygen saturation.

PSG provides a reference for evaluating wearable estimates.

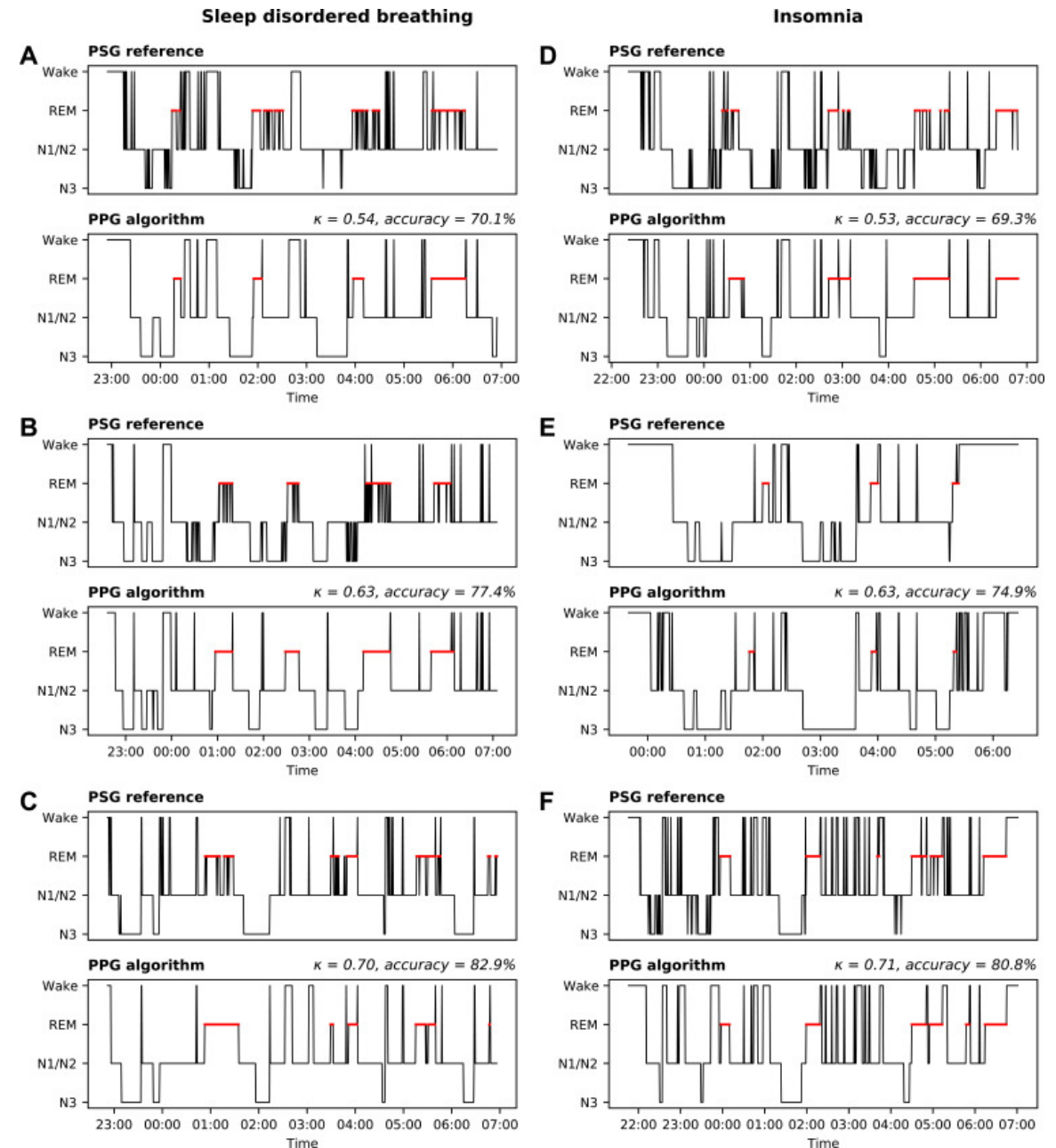


# Compare Sleep Labels over Time

Compare each PSG trace with its model estimate from PPG signal alone.

Current PPG models can only do a rough estimation.

**Multimodal models** combine signal types: motion, pulse/HRV, oxygen saturation ( $\text{SpO}_2$ ), or skin temperature. Inputs are model-specific.



# Digital Phenotyping Connects Everyday Patterns

**Digital phenotyping** quantifies behavioral and physiological patterns in everyday life using data from personal digital devices.

| Measurements               | Pattern to describe                         |
|----------------------------|---------------------------------------------|
| Motion and step counts     | When activity occurs and how it changes     |
| Resting heart rate and HRV | Changes under comparable resting conditions |
| Sleep duration and timing  | Sleep amount and schedule                   |

Fewer steps, later sleep, and changed resting heart metrics can have several causes. Remember that the pattern alone is not a diagnosis, but it indicates some shared characteristics among people in everyday life.

# Break

---

# Build with Your Own Data

- You can borrow a **Fitbit Air**. Scan the QR code if you have not completed the form.
- Borrowing is optional. **Synthetic data** provide a complete path for the labs and project, with the same grading criteria.



# Time Processing and Data Quality

# The Time Index Reveals Broken Cadence

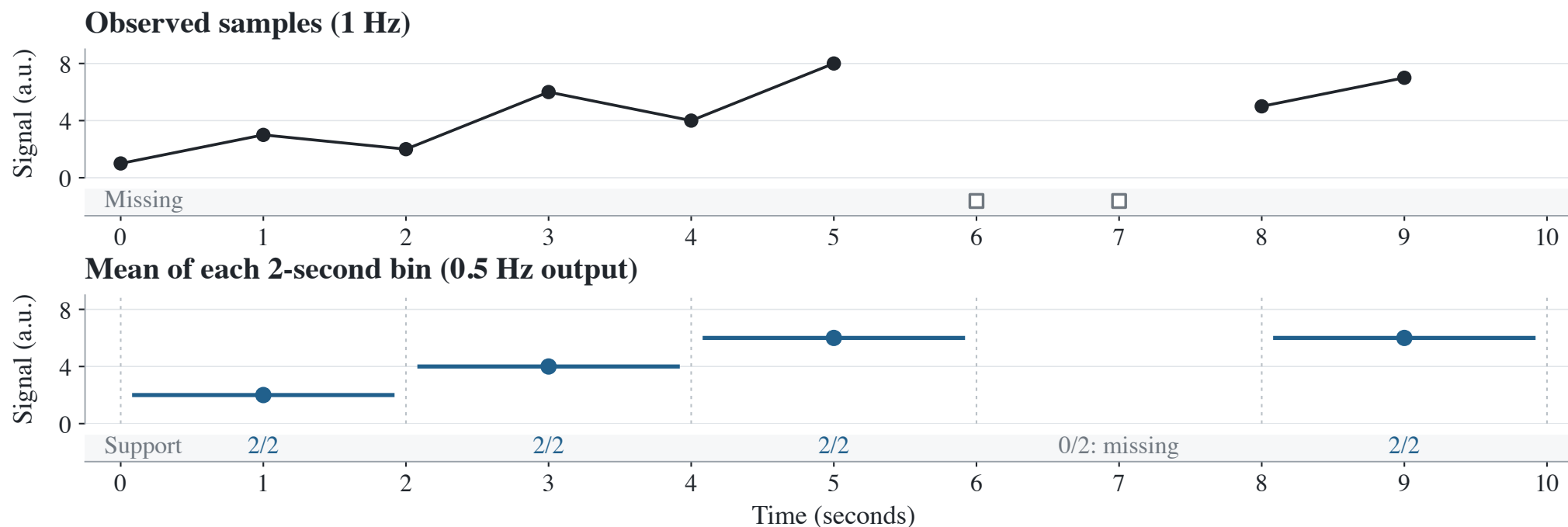
```
timestamp      ax      ay      az
12:00:00.000   0.01   -0.03   1.00
12:00:00.020   0.04    0.11   0.98
12:00:00.040   0.19    0.37   0.91
12:00:00.110   0.06    0.08   1.01
Observed gaps: 20 ms, 20 ms, 70 ms
50 Hz = 50 samples/s -> nominal gap: 20 ms
```

A **time index** associates each sample with its timestamp.

- The 70 ms gap could mean missing samples, clock jitter, buffering, or a pause
- A fixed-rate filter no longer receives the input cadence it assumes

# Downsampling Uses a Coarser Grid

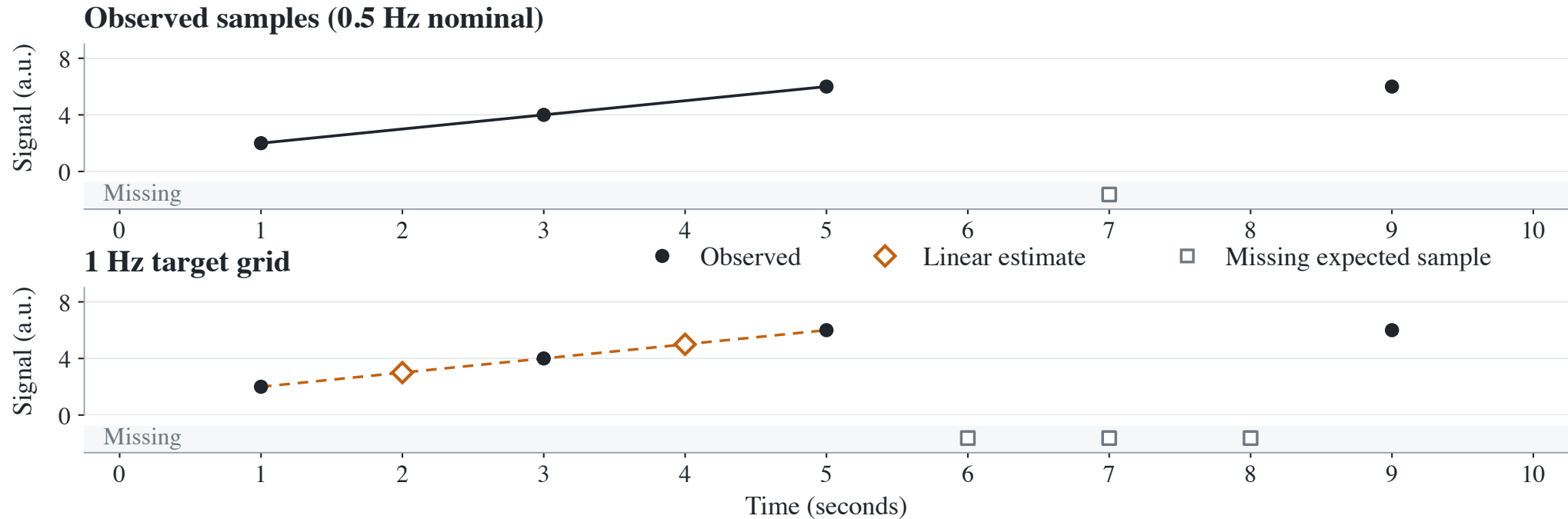
**Downsampling** uses fewer samples at longer intervals: here, means of 2-second bins.



First mean:  $(1 + 3) / 2 = 2$ . No samples in  $[6, 8)$ : keep that bin missing.

# Upsampling Uses a Denser Grid

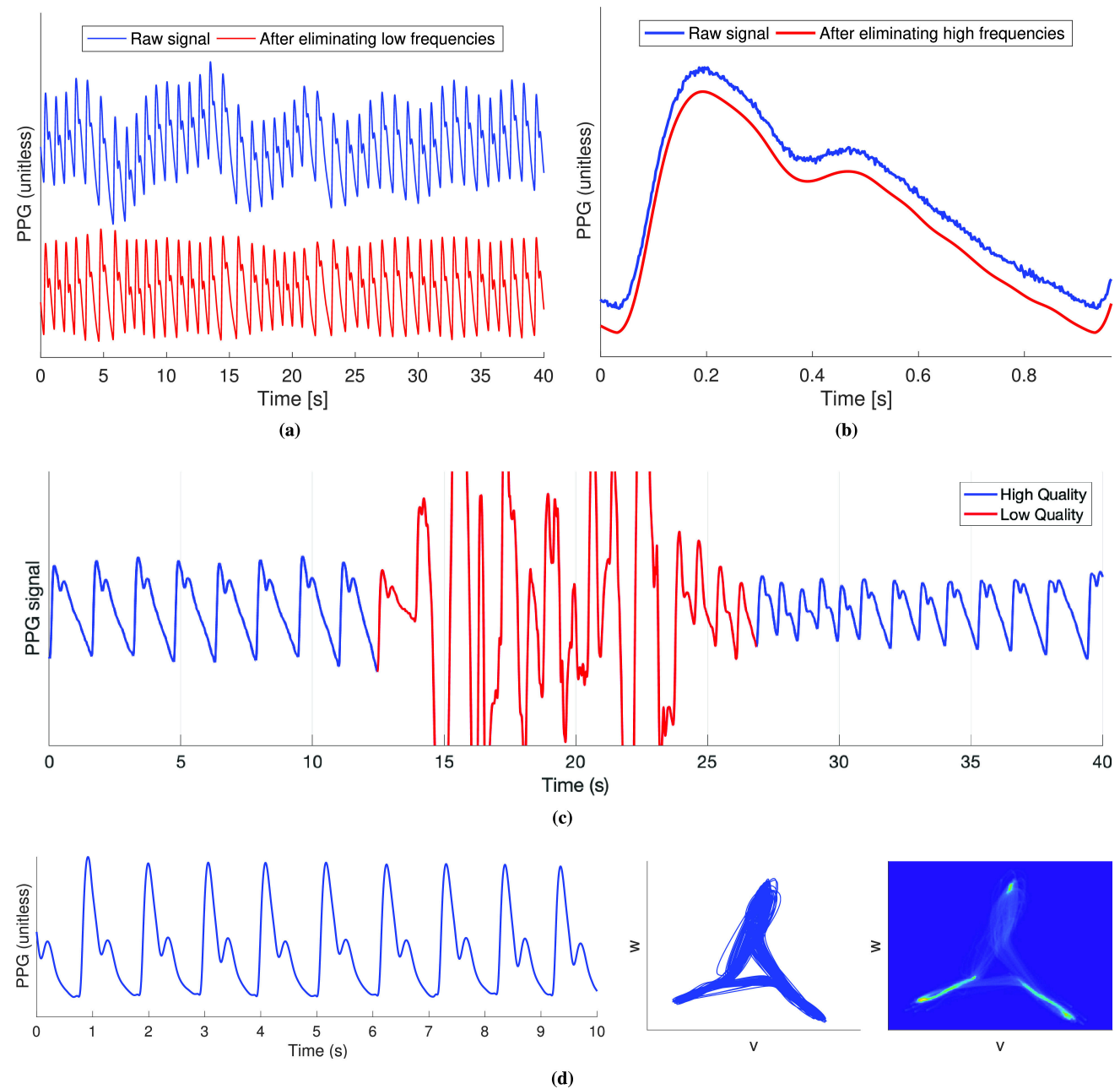
**Upsampling** adds points to a finer time grid. New grid points are not new observations.



Orange hollow points are modeled; the 2-second gap limit leaves 6–8 s missing.

# Quality Control Comes Before Features

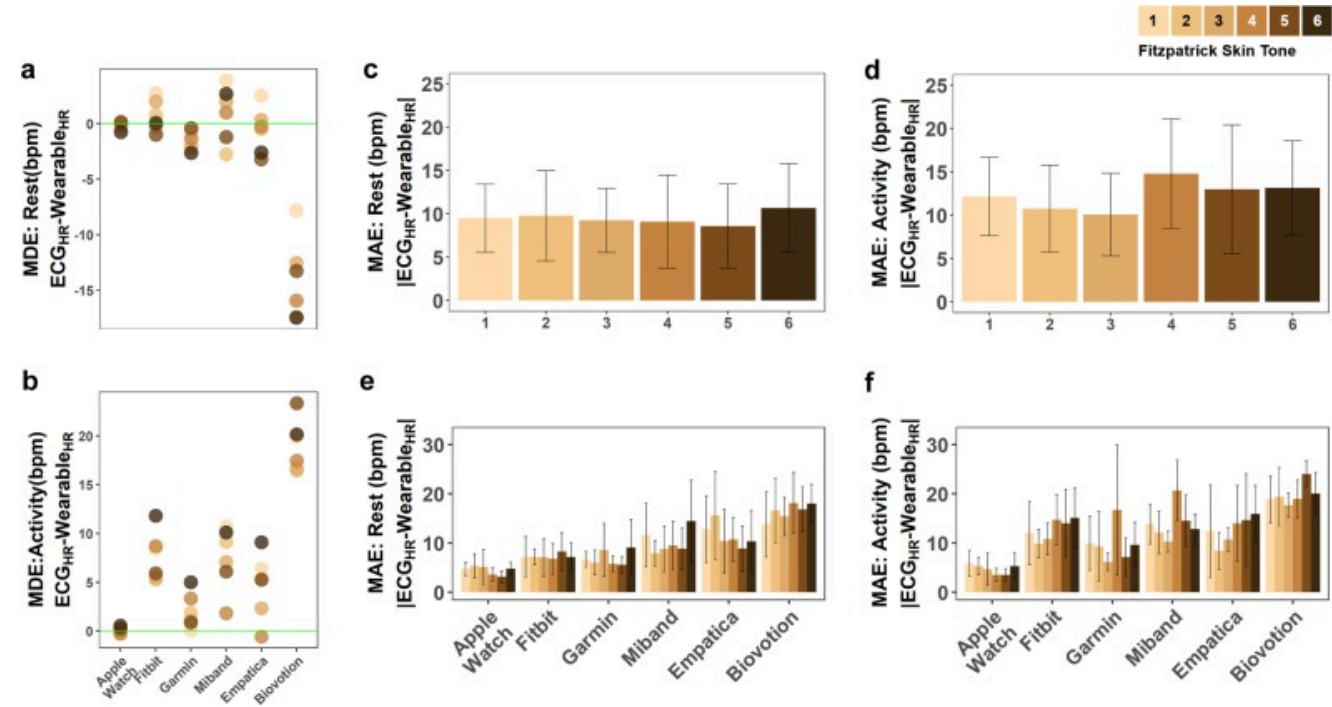
- Filtering targets baseline wander and out-of-band noise
- Signal-quality assessment can reject unusable windows
- Detected landmarks, such as pulse peaks, support rate and shape features
- Parameters and rejected windows remain part of provenance



# Error Changes with Activity and Device

- Error varied across devices and activity conditions in this study
- Activity increased error on average
- Fit, placement, perfusion, hardware, and algorithms can matter
- An exercise claim needs evaluation during exercise

Evaluate a specific **metric, device, population, and context**. Evidence about clinical pulse oximetry cannot be transferred automatically to consumer wrist pulse rate.

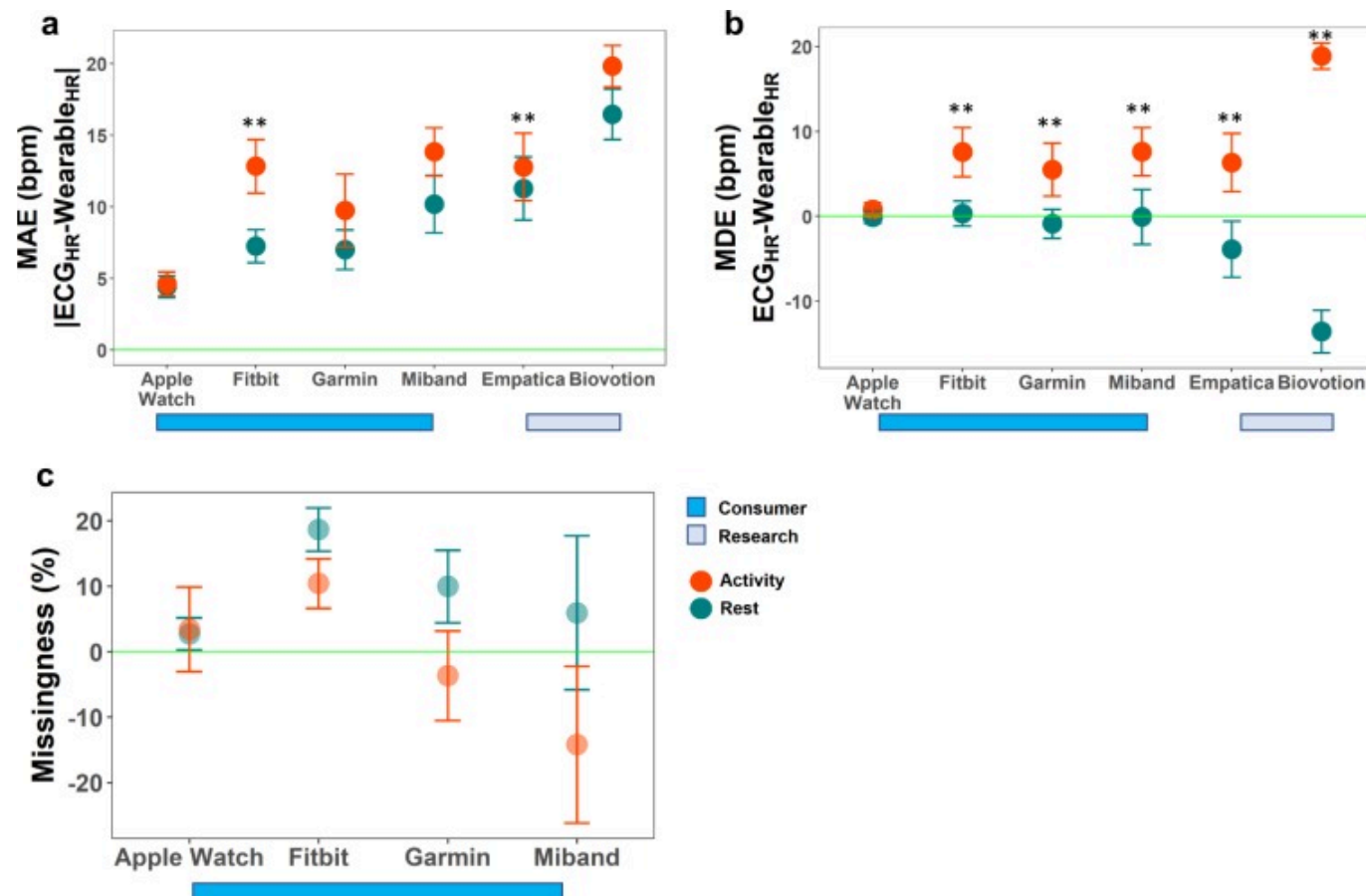


# Missingness Is Device- and Activity-Specific

**Coverage:** fraction of expected windows with a usable value.

48 usable / 60 expected = 80%

- In this study, devices and activity affected error and missingness
- Accuracy compares available values with a reference
- Good accuracy can coexist with poor coverage



# A Daily Join Keeps Three Sources Visible

**Analysis grain:** one person-day in local time · 2026-09-18

| Clinical/EHR                                                                            | Everyday sensor                                                                         | Self-report                                                                                     |
|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------|
| <b>Hemoglobin:</b> 10.8 g/dL<br>Specimen: 09:10<br>Status: final<br>Observation/obs-401 | <b>Resting HR:</b> 75 bpm<br>Daily estimate<br>Wear coverage: 18.5 h<br>Method: DERIVED | <b>Fatigue:</b> 7/10 at 21:05<br>Recall: “today”<br>Exact item retained<br>Note: “Slept poorly” |

The date aligns records for analysis.

But always remember: It does **not** make them simultaneous or show that one caused another.

# Evaluate the Exact Claim

Complete this sentence before choosing a metric:

How accurately and consistently does **device X** estimate **metric Y** for **population Z** during **context A**, compared with **reference R**, at **coverage C**?

Then measure the failures that matter: misses, false detections, absolute error, coverage, and subgroup performance.

# Evaluating Wearable Metrics

After defining the claim, follow this sequence:

1. **Inspect the records:** preserve source, time, units, and record grain.
2. **Specify the processing:** declare filters, windows, thresholds, and model versions.
3. **Check usable coverage:** report gaps and rejected windows beside available values.
4. **Evaluate the claim:** compare with an appropriate reference for the population and context.
5. **Report the result:** keep supporting evidence, alternatives, and uncertainty visible.

Keep a record of these choices as you work.

# Wearable Data Access: APIs and OAuth

Connect your code to synced wearable records through an API and scoped authorization.

# Google Health API Reads Synced Records

Fitbit → phone sync → Google Health account in the cloud

Notebook ← authorized API request ← JSON records from your Google Health account in the cloud

The Google Health API lets your code retrieve Fitbit steps, heart rate, and sleep records for analysis. Sync puts records in your account. App consent determines which records the notebook may request.

# Register the App, Then Authorize Access

OAuth is delegated authorization: a third-party **app/client** (registered software) gets limited access without your password.

We registered BINF 4070 Personal Health Data Access as the third-party app for this course.

1. For your own app, register a client with Google. Google issues the client ID; you configure and register the callback URL (return address).
2. Each user grants scopes (permissions). App registration alone grants no access to student data.
3. Google returns a code to the registered callback. The helper exchanges it using the same client ID and matching callback URL for tokens.
4. The app (or the notebook in our lab) presents an access token (temporary credential) to request permitted records.

# A Representative Needs Your Permission

Let's use an analogy to explain OAuth:

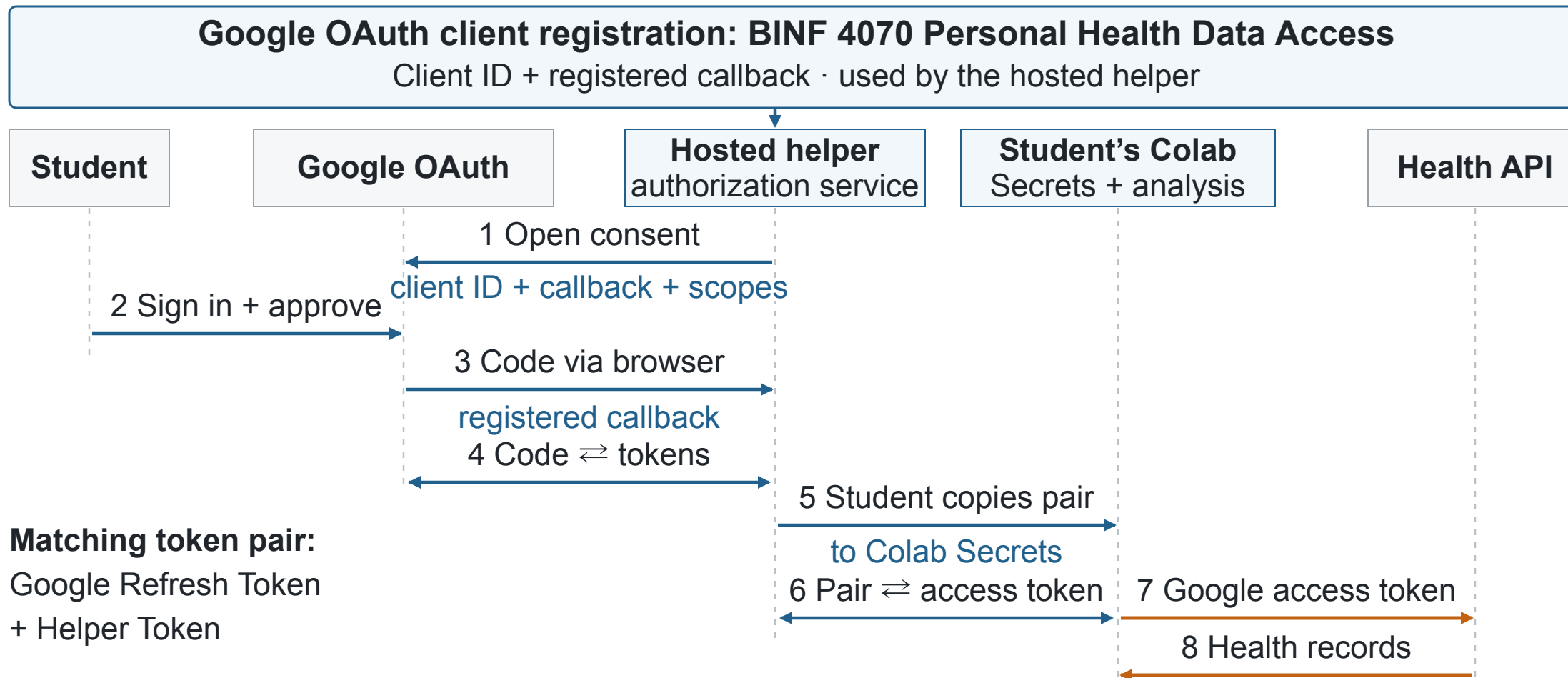
You are a hotel guest (Hotel is Google). Your personal health data sits in your hotel room, and a third-party app is a representative acting for you.

1. **You approve** which items the representative may access (**scopes**).
2. **Google's front desk** sends a temporary receipt (**code**) to the representative's registered return address (**callback**).
3. **The representative exchanges the receipt** for a limited, expiring pass (**access token**).
4. **The representative presents the pass** to retrieve the permitted data.

Registering the representative is not the user's permission. It's the third-party's responsibility to register a solid and secure application (as the representative).

You (as the user) authorize access **without giving the app your Google password**.

# Course OAuth Flow



When you want to provide service to your users: You will be the one who registered the third-party app and host the helper!

# The Course Helper Renews Access

The hosted helper uses the course app's registered identity for authorization and token renewal.

The notebook needs to run separately, that's why you are getting these tokens so that you can use them in your own notebook.

In real-world practice, you don't need to give these token back to your users.

Each person grants access separately: Alice's tokens access Alice's data; Bob's access Bob's.

Save your matching pair in Colab Secrets:

| Secret                      | Purpose                                 |
|-----------------------------|-----------------------------------------|
| GOOGLE_HEALTH_REFRESH_TOKEN | Refresh token: renews Google access     |
| GOOGLE_HEALTH_HELPER_TOKEN  | Course credential for your helper calls |

# DataPoint Wraps One Metric Payload

```
{
  "dataSource": {
    "recordingMethod": "PASSIVELY_MEASURED",
    "platform": "FITBIT"
  },
  "heartRate": {
    "sampleTime": {
      "physicalTime": "2026-09-18T14:32:00Z",
      "utcOffset": "-14400s"
    },
    "beatsPerMinute": "78"
  }
}
```

## Read from the envelope inward

- `dataSource` records origin and recording method
- One typed payload—here `heartRate`—carries the metric
- The payload defines its own value and time shape
- Other payloads include `steps`, `sleep`, and daily derived metrics

# Build with Your Own Data

- You can borrow a **Fitbit Air**. Scan the QR code if you have not completed the form.
- Borrowing is optional. **Synthetic data** provide a complete path for the labs and project, with the same grading criteria.



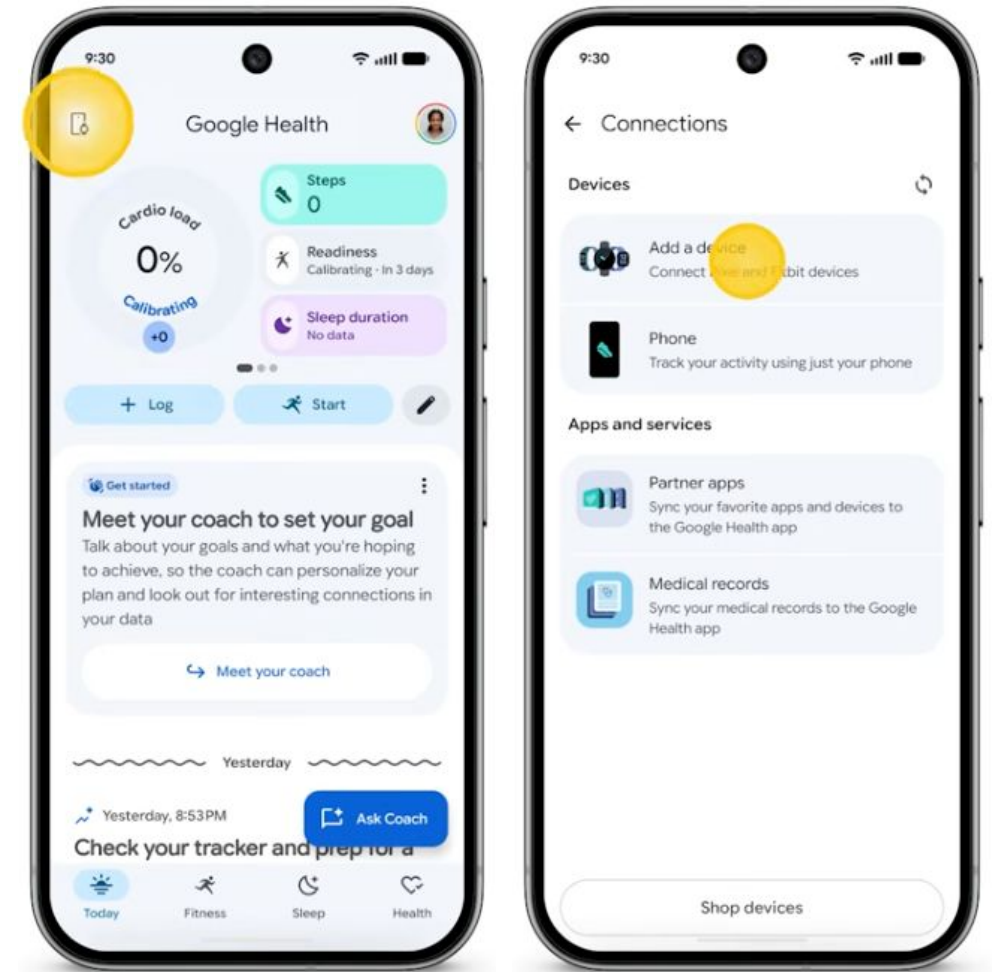
# Lab: Connect and Explore Wearables

Pair Fitbit Air, sync Google Health, then retrieve and visualize one day of wearable records.

# Prepare Fitbit Air in Google Health

1. Put **Fitbit Air on its charger** near your phone.  
Install or update **Google Health**.
2. Choose **Sign in with Google** and use a compatible **Google Account**.
3. Open **Connections** (top-left icon), then **Add a device**.

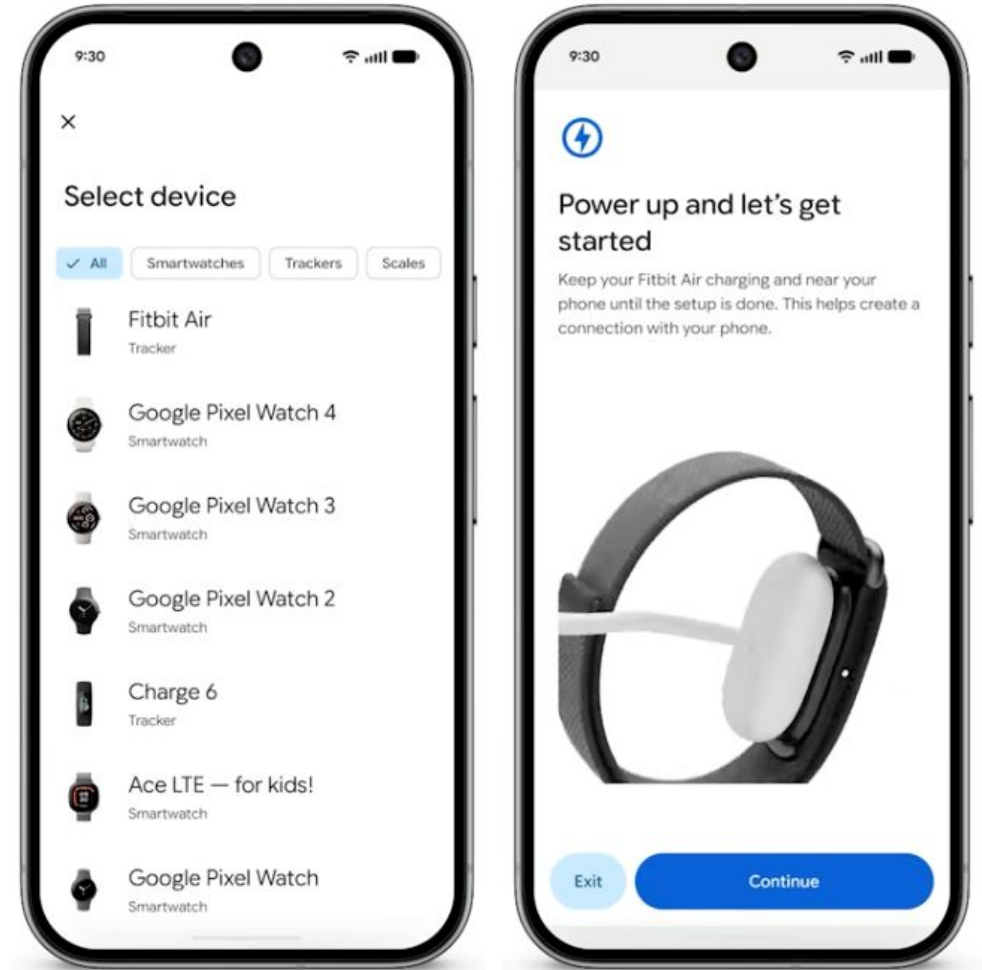
Use the same account for Google Health and course authorization. **API consent follows separately in the notebook.**



# Pair Fitbit Air and Check Sync

4. In **Select device**, choose **Fitbit Air**.
5. Keep Air charging nearby. Tap **Continue**, then **Pair** when prompted; complete the phone's setup prompts.
6. Under **Connections** → **Devices**, check that Air appears with a recent **last-synced time** and **battery level**.

Newly issued devices may lack earlier heart-rate or sleep records. **Sample data support today's lab.**



# Lab: Retrieve, Plot, and Summarize

Use synthetic data by default; Orson demonstrates live Fitbit data.

1. Fitbit API setup or sample path
2. Retrieve data; choose a step source
3. Create and inspect simple plots
4. AI summary and evidence check using your OpenAI API key

Thanks for the Attention  
Q&A